Computer Vision · Week 1

Beginner study guide

Day 1 – Images, pixels & colour spaces
Day 2 – Image transformations
Day 3 – Filtering, convolution & edge detection

Python/NumPy basics · Notes · Diagrams · Practice quiz with answers · Cheat sheet
Runnable code is in the **code/** folder next to this PDF.

00 · Just enough Python & NumPy for Week 1

You don't need to be a programmer. The lectures use **three libraries**, and you only need a small piece of each.

Library	Imported as	Job in plain words
OpenCV	`import cv2`	The image toolbox: load, convert colours, rotate, blur, find edges…
NumPy	`import numpy as np`	Stores grids of numbers (an image *is* one) and does maths on them
Matplotlib	`import matplotlib.pyplot as plt`	Draws/shows pictures and charts on screen

---

1. A NumPy array = a grid of numbers

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6]])
print(a.shape)   # (2, 3)  -> 2 rows, 3 columns
print(a[0, 2])   # 3       -> row 0, column 2  (counting starts at 0!)
print(a[1])      # [4 5 6] -> the whole of row 1
```

- `.shape` tells you the size: `(rows, columns)`, or `(rows, columns, channels)` for colour images.
- **Counting starts at 0.** The first row is row 0.
- `a[row, col]`: the row always comes first.

2. Slicing = cutting out a piece

`start:stop` means "from start up to **but not including** stop".

```
img[100:200, 50:300]   # rows 100..199, columns 50..299  -> a crop
img[:, 0]              # ":" alone means "all" -> every row, column 0
img[0:60, 0:60] = 255  # set a whole block at once (paint it white)
```

3. Data types: `uint8` and `float64`

- Images are usually `uint8`: whole numbers from **0 to 255** only.
- Going past 255 **wraps around** in NumPy: `250 + 10 = 4` (!). OpenCV's `cv2.add` **clips** instead: `250 + 10 = 255`.
- Maths that can go negative (edge detection) needs `float64` (`cv2.CV_64F` in OpenCV).
- `np.float32([...])`: OpenCV's transformation functions want their matrices as 32-bit decimals. That's why the lectures write `np.float32([[1,0,100],[0,1,50]])`.

4. Useful NumPy bits seen in the lectures

```
np.zeros((3, 3))          # 3x3 grid of 0s
np.ones((3, 3)) / 9       # 3x3 grid of 1/9  (an "average" kernel)
np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])  # the sharpening kernel
a * b                     # multiply element by element (same-size grids)
np.sum(a)                 # add everything up
```

5. Showing an image with Matplotlib

```
plt.imshow(img_rgb)            # colour image (must be RGB order!)
plt.imshow(gray, cmap='gray')  # 1-channel image: tell it to use grey shades
plt.title("My title")
plt.axis('off')                # hide the x/y ruler numbers
plt.show()                     # actually display it

plt.subplot(1, 2, 1)           # "1 row, 2 columns, draw in slot 1"
plt.subplot(1, 2, 2)           # "...now slot 2"  -> side-by-side pictures
```

> Without `cmap='gray'`, a 1-channel image is drawn in yellow-green-purple colours (matplotlib's default colour map). It's the same data, just painted with fake colours.

6. Python bits used in the mini-projects

```
def image_viewer(img_path, mode="original"):   # a function with a default value
    if mode == "gray":
        ...
    elif mode == "edges":       # "else if"
        ...

f"Mode: {mode}"                # f-string: puts the variable's value into the text

for i, cnt in enumerate(contours):   # loop that also gives the counter i = 0, 1,
2...
    ...

filters = {"blur": ..., "sharpen": ...}   # dictionary: name -> value
for name, output in filters.items():      # loop over (name, value) pairs
    ...

x = a if condition else b                 # one-line if/else
```

That's all the Python you need for Week 1. Next: `day1_images_and_colors.md`.

Day 1 · Introduction to Computer Vision

Lecture covers: What is CV? · Images as data · Colour spaces · Transformations (overview) · Filtering (intro) · Applications · Mini-project: Image Viewer

Code for today: `code/day1_01` → `day1_04`

---

1. What is Computer Vision?

> **Computer Vision (CV)** is the part of AI that lets computers *see*: take in pictures or video and work out what's in them.

Your eyes just see a cat. A computer receives **millions of numbers** and has to work out from those numbers that there's a cat. CV is the set of tricks for doing that.

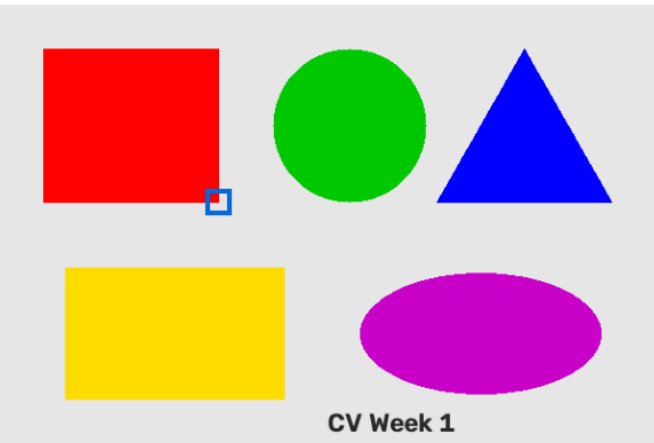
Applications the lecture lists (learn these for the eval):

Area	Example
Self-driving cars	Detect lanes, cars, pedestrians, traffic signs
Medical imaging	Find tumours in X-rays/MRI scans
Surveillance	Spot people or unusual activity on CCTV
Augmented Reality (AR)	Snapchat filters, Pokémon GO (stick graphics onto the real world)
Traffic monitoring	Count / detect cars
Retail	Face recognition, product tracking on shelves
Creative AI	Art generation

---

2. Images as data

What you see: a picture



What the computer sees: numbers (0 = black, 255 = white)
8×8 zoom of the blue box (corner of the red square), in grayscale

76	76	76	76	76	230	230	230
76	76	76	76	76	230	230	230
76	76	76	76	76	230	230	230
76	76	76	76	76	230	230	230
76	76	76	76	76	230	230	230
230	230	230	230	230	230	230	230
230	230	230	230	230	230	230	230
230	230	230	230	230	230	230	230

- A digital image is a **grid of tiny squares called pixels** (short for "picture elements").
- Each pixel holds **numbers that say how bright it is**.
- **Grayscale image:** 1 number per pixel. `0 = black`, `255 = white`, in between = grey.
- **Colour image:** 3 numbers per pixel, one each for **Red, Green, Blue**.
- Why 0–255? Each number is stored in **8 bits** (1 byte), and 8 bits can hold $2^8 = 256$ different values. The type is called **`uint8`** (unsigned 8-bit integer).
- **Resolution** = how many pixels the image has (width × height). More pixels means more detail. The lecture's photo was **5472 × 3648 ≈ 20 million pixels**.

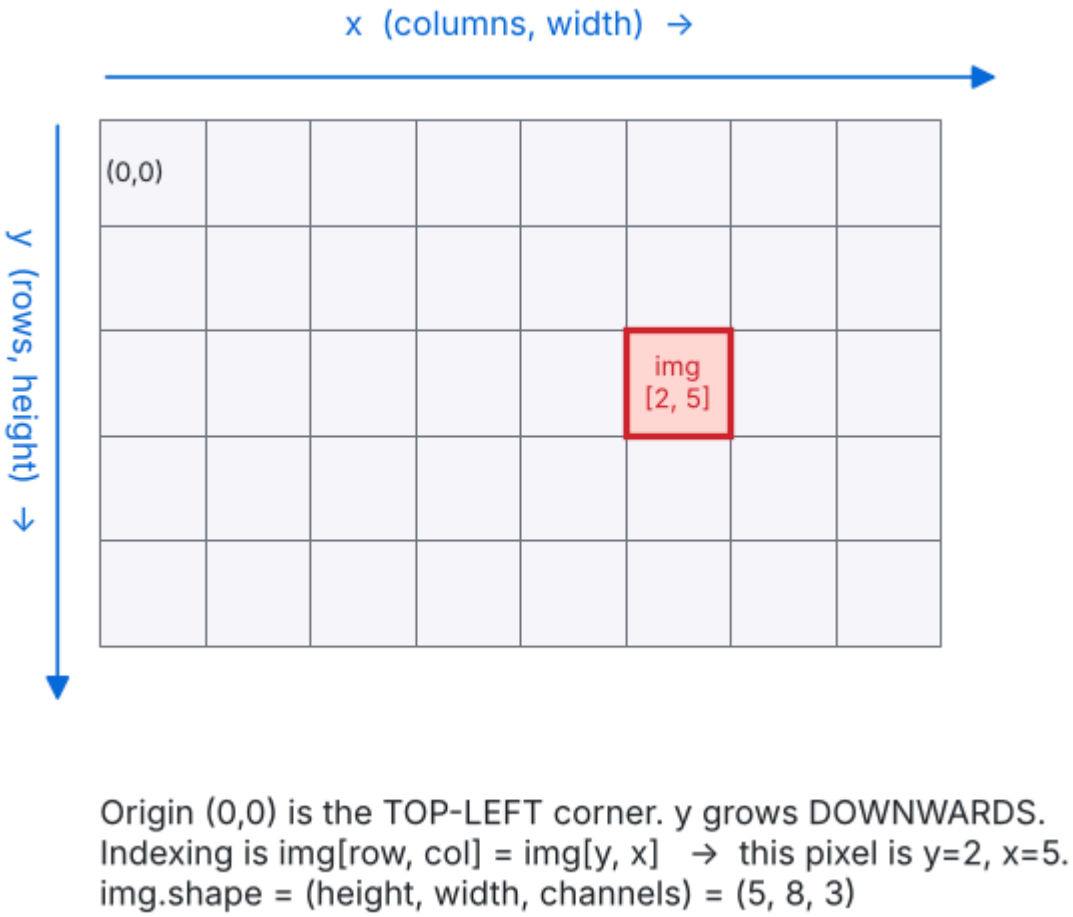
Shape: the most important 3 numbers

```
img = cv2.imread('sample.jpg')
print(img.shape)        # (5472, 3648, 3)  in the lecture
#                          height  width  channels
```

Position	Meaning	Other name
`shape[0]`	**height**, the number of rows	`rows`
`shape[1]`	**width**, the number of columns	`cols`
`shape[2]`	**channels** (3 for colour; missing for grayscale)	

> ⚠️ Shape is **(height, width)**, but OpenCV functions that ask for a *size* want **(width, height)**. E.g. `cv2.warpAffine(img, M, (cols, rows))`. This flip catches everyone at least once.

Coordinates: the origin is the TOP-LEFT



- `(0, 0)` is the **top-left** corner (not the bottom-left like in school maths graphs).
- **x** goes right (columns). **y** goes **down** (rows).
- To read a pixel: `img[y, x]`, i.e. **row first**.

```
print(img[50, 50])     # [172 169 161]  in the lecture → B=172, G=169, R=161 (a light grey)
```

Why grey? When all three numbers are roughly equal, the colour is grey. All 0 is black, all 255 is white.

The BGR trap ⚠️ (very likely to be asked)

- OpenCV loads colour images in **B, G, R** order (Blue first). That's a historical choice.
- Matplotlib (and almost everything else) expects **R, G, B**.
- So you **must convert before showing with matplotlib**:

```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb)
```

- If you forget, **red and blue swap**. The lecture's *Mode: original* screenshot in the mini-project shows exactly that: the orange Pantone swatches came out **blue**, because the function passed the BGR image straight to `plt.imshow`.

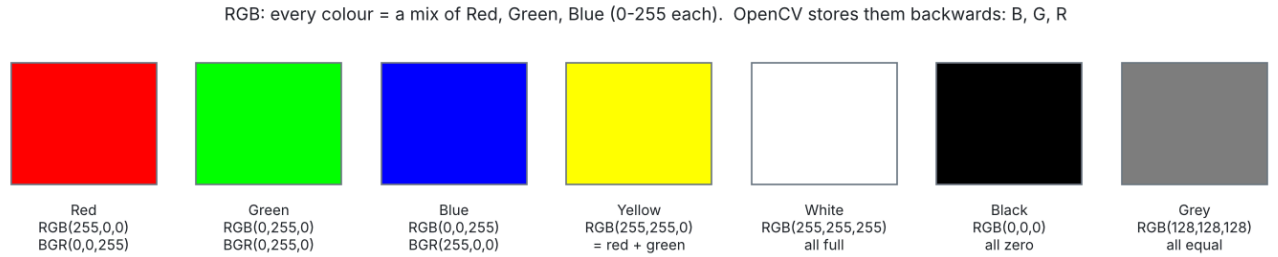
- `cv2.imshow()` (OpenCV's own window) expects BGR, so it needs **no** conversion.

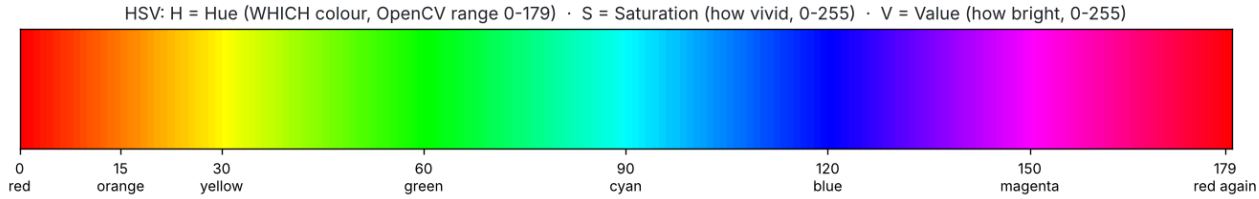
▶ Run `code/day1_01_load_and_inspect.py` and `code/day1_02_pixels_by_hand.py`.

---

3. Colour spaces

A **colour space** is just a *way of describing colour with numbers*. Same colour, different description.





RGB (and OpenCV's BGR)

- Every colour = a mix of **Red + Green + Blue** light, each 0–255.
- Red + Green = Yellow. All full = White. All zero = Black.
- Problem: if a red ball goes into shadow, **all three** numbers change. That makes "find the red things" awkward.

Grayscale

- One brightness number per pixel. 3× less data, and enough for edges and shapes.
- OpenCV's formula: **Gray = 0.299·R + 0.587·G + 0.114·B**. Green counts most because human eyes are most sensitive to green.
- `gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)` → shape becomes `(h, w)` with no channel number.

HSV = Hue, Saturation, Value

Think of describing paint to a friend:

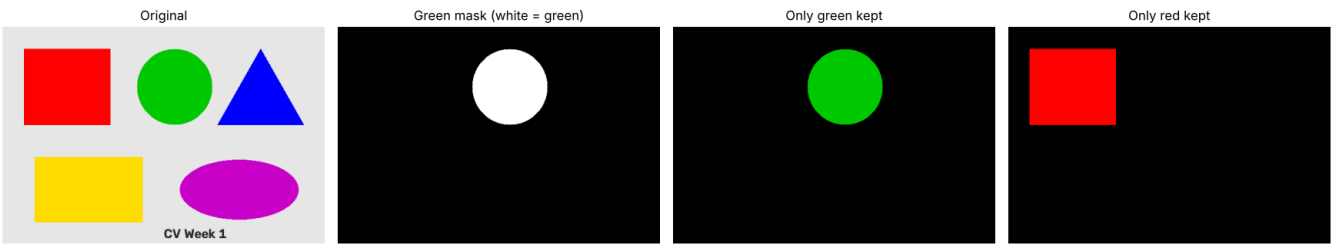
Letter	Means	Plain words	OpenCV range
Hue	*Which* colour	"it's red / green / blue" (position on the colour wheel)	**0–179** (half of 0–360°, so it fits in a byte)
Saturation	How *pure / vivid*	0 = washed-out grey, 255 = vivid	0–255
Value	How *bright*	0 = black, 255 = bright	0–255

Why HSV helps with colour detection: the *colour* (Hue) is stored separately from the *brightness* (Value). A red ball in sunlight and in shadow keeps roughly the **same Hue**, so "find red" becomes "find Hue ≈ 0". That's why the lecture says converting between colour spaces "helps in tasks like colour detection".

```
hsv  = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, (40, 80, 50), (80, 255, 255))   # white where the pixel is
green
only_green = cv2.bitwise_and(img, img, mask=mask)
```

- A **mask** is a black-and-white image: white (255) = "yes, keep this pixel", black (0) = "no".
- Red sits at **both ends** of the hue scale (0 and 179), so red needs two ranges joined with `|`.

Lecture note: the lecture converts to HSV and then *back* to RGB before showing it, so the "HSV Color Space" picture looks exactly like the original. To actually *see* HSV, show its H, S and V channels separately, as `day1_03_color_spaces.py` does.



▶ Run `code/day1_03_color_spaces.py`.

---

4. Transformations & filtering: Day 1 preview

Day 1 only *introduces* these. They're covered properly on Days 2 and 3.

- **Translation**: shifting the image. **Scaling**: resizing. **Rotation**: turning around a pivot. **Perspective**: changing the viewpoint, as if photographed from another angle.
- **Convolution** = sliding a small grid of numbers (a **kernel**) over the image. Filters made this way detect **edges, textures and patterns**, and this is the **foundation of CNNs** (Convolutional Neural Networks, the deep-learning models used for images).
- Day 1 already shows the sharpening kernel `[[0,-1,0],[-1,5,-1],[0,-1,0]]` and `cv2.Canny(img, 100, 200)` for edges.

---

5. Mini-project: Image Viewer

Task: load an image → convert to grayscale → apply edge detection → toggle between modes.

```
def image_viewer(img_path, mode="original"):
    img = cv2.imread(img_path)
    if mode == "gray":
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    elif mode == "edges":
        img = cv2.Canny(img, 100, 200)
    plt.imshow(img, cmap='gray' if mode != "original" else None)
    plt.title(f"Mode: {mode}")
    plt.axis('off')
    plt.show()
```

Read it line by line: - `mode="original"` is a **default**: if you don't pass a mode, you get "original". - Gray and edges results are **1-channel**, so they need `cmap='gray'`. - **Bug:** in "original" mode it shows a **BGR** image with matplotlib, which swaps red and blue. Fix: `img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` in an `else:` branch.

▶ Run `code/day1_04_mini_project_image_viewer.py`. It also has an interactive keyboard version for your own computer.

✅ Day 1: check yourself

1. What does `img.shape = (480, 640, 3)` mean? *(480 rows high, 640 columns wide, 3 colour channels)*
2. Which pixel is `img[10, 200]`? *(row 10 (y = 10), column 200 (x = 200))*
3. Why do colours look wrong in matplotlib after `cv2.imread`? *(OpenCV uses BGR, matplotlib expects RGB)*
4. Why is HSV better than RGB for detecting a coloured object? *(Hue holds the colour separately from the brightness, so shadows don't change it much)*
5. What range does Hue have in OpenCV? *(0–179)*

More questions: `practice/quiz.md`.

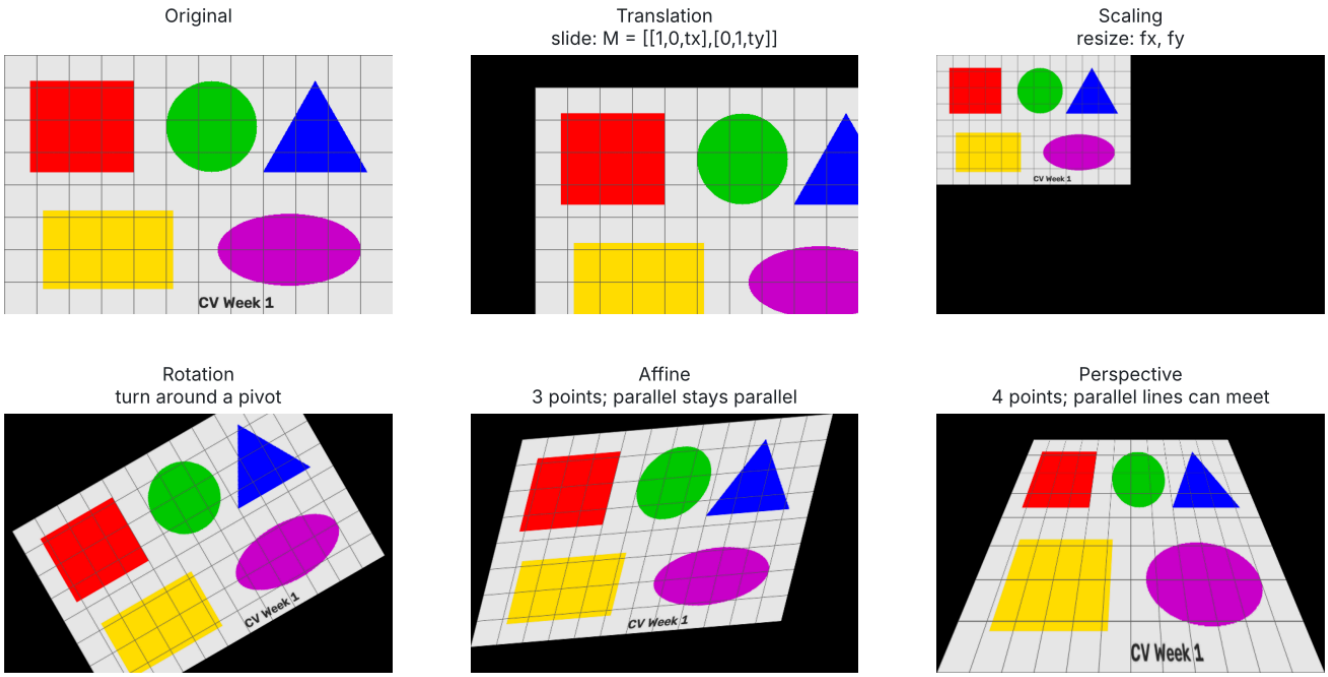
Day 2 · Image Transformations

Lecture covers: Translation · Scaling (resizing) · Rotation · Affine transformations · Perspective transformations · Mini-project: Image Augmentation Tool

Code for today: `code/day2_01` → `day2_03`

The big picture

A **transformation** moves pixels to **new positions**. The colours don't change, only *where* each pixel ends up.



Almost all of them follow the same 2-step recipe in OpenCV:

```
M   = <make a small matrix that describes the movement>
out = cv2.warpAffine(img, M, (width, height))      # or cv2.warpPerspective for 4-point warps
```

Think of **M** as a recipe card that says, for every pixel, "you started at (x, y), now go to (x', y')". `warpAffine` follows the recipe for every pixel.

⚠️ The lecture's Day 2 snippets use `rows` and `cols` without defining them first. You need:

```
rows, cols = img.shape[:2]     # rows = height, cols = width
```

---

1. Translation: sliding the picture

Shifting the image along x (right/left) and/or y (down/up).

```
M = np.float32([[1, 0, 100],     # x' = 1·x + 0·y + 100   → move 100 px RIGHT
                [0, 1,  50]])    # y' = 0·x + 1·y +  50   → move  50 px DOWN
translated = cv2.warpAffine(img, M, (cols, rows))
```

- General form: `[[1, 0, tx], [0, 1, ty]]`. `tx` is the horizontal shift, `ty` the vertical shift.
- **Negative** values move left/up.
- The area that becomes empty is filled with **black**, and anything pushed past the edge is cut off. That's why the lecture's "Translated Image" has a black strip on the top and left.
- The `1`s keep the size the same; the `0`s mean "x doesn't affect y and vice versa".

---

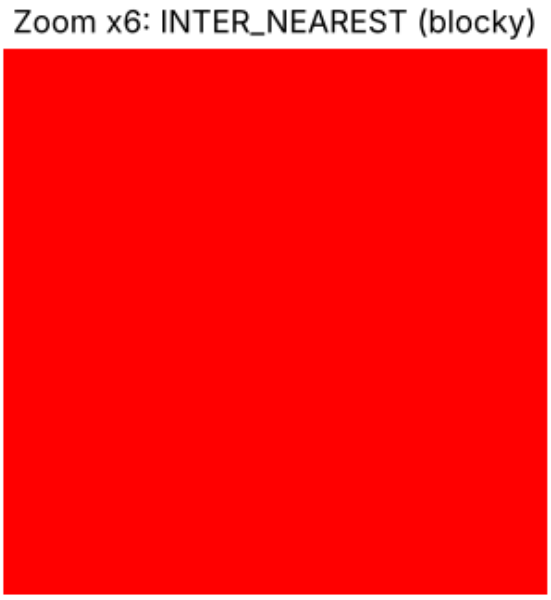
2. Scaling: resizing

```
scaled = cv2.resize(img, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_LINEAR)
```

- `fx` = horizontal factor, `fy` = vertical factor. `0.5` = half, `2` = double.
- `None` = "I'm not giving an exact size, work it out from fx/fy". Or give the size directly: `cv2.resize(img, (300, 200))`. Remember that's **(width, height)**.
- Different fx and fy **stretch or squash** the image (the aspect ratio changes).

Interpolation = how to fill in pixels that didn't exist before (enlarging) or merge ones that get squeezed together (shrinking):

Flag	How	Use it when
`INTER_NEAREST`	Copy the nearest pixel	Fastest, blocky look
`INTER_LINEAR`	Average of the 4 nearest	Default, good all-rounder
`INTER_CUBIC`	Uses 16 nearby pixels	Smoother when **enlarging** (slower)
`INTER_AREA`	Averages areas	Best for **shrinking**





---

3. Rotation: turning around a pivot

```
M = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 1)   # (centre, angle, scale)
rotated = cv2.warpAffine(img, M, (cols, rows))
```

Argument	Meaning
`center`	The pivot point (x, y). `(cols/2, rows/2)` = the middle of the image
`angle`	Degrees. **Positive = anti-clockwise**, negative = clockwise
`scale`	`1` = same size. `0.6` = shrink while rotating, which keeps the corners inside

- The output canvas stays the same size, so **the corners get cut off** and black triangles appear (see the lecture's "Rotated Image (45°)").
- The matrix OpenCV builds is `[[cos θ, sin θ, …], [−sin θ, cos θ, …]]`, scaled. For 45°, cos = sin ≈ 0.707.
- Shortcuts that don't crop: `cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)`. Mirror with `cv2.flip(img, 1)`.

---

4. Affine transformation: 3 points

> **Affine = keeps parallel lines parallel.** Straight lines stay straight, and lines that were parallel stay parallel. Squares can become slanted parallelograms, but never trapezoids.

Translation, scaling, rotation and **shear** (slanting, like italic text) are *all* affine. An affine transform is defined by **3 points before → 3 points after**:

```
pts1 = np.float32([[50, 50], [200, 50], [50, 200]])    # 3 points in the original
pts2 = np.float32([[10, 100], [200, 50], [100, 250]])  # where those 3 should go
M = cv2.getAffineTransform(pts1, pts2)                 # → 2×3 matrix
affine = cv2.warpAffine(img, M, (cols, rows))
```

Why 3? The affine matrix has **6 unknown numbers** (2×3). Each point pair gives 2 equations (one for x, one for y), so 3 pairs give the 6 equations needed.

---

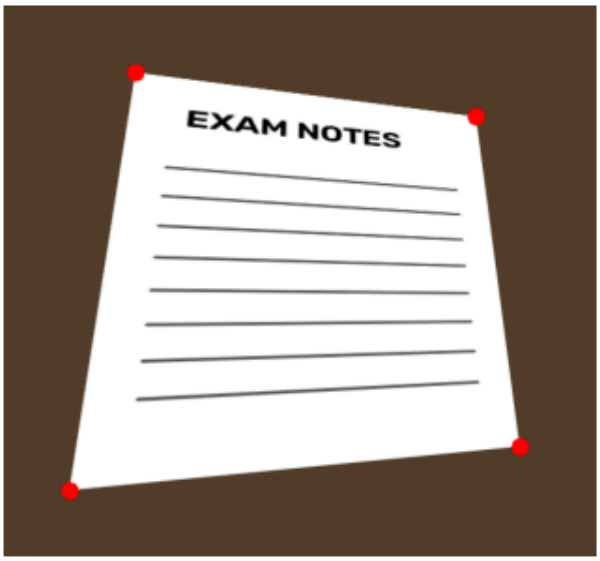
5. Perspective transformation: 4 points

> **Perspective maps any 4-sided shape to any other 4-sided shape.** Parallel lines can **meet**, like train tracks going into the distance. Straight lines still stay straight.

```
pts1 = np.float32([[56,65],[368,52],[28,387],[389,390]])   # 4 corners in the photo
pts2 = np.float32([[0,0],[300,0],[0,300],[300,300]])       # where they should go
M = cv2.getPerspectiveTransform(pts1, pts2)                # → 3×3 matrix
perspective = cv2.warpPerspective(img, M, (300, 300))
```

- Needs **4 point pairs** (the 3×3 matrix has 8 free numbers, and 4 pairs × 2 = 8 equations).
- Points must be listed in the **same order** in both lists (here: top-left, top-right, bottom-left, bottom-right).
- Uses `warpPerspective`, not `warpAffine`.
- **Main use, document scanning:** a page photographed at an angle is a tilted 4-sided shape. Map its 4 corners to a neat rectangle and you get a flat "scan". (Apps like CamScanner and Adobe Scan do this.)

Photo of paper (red = 4 corners)



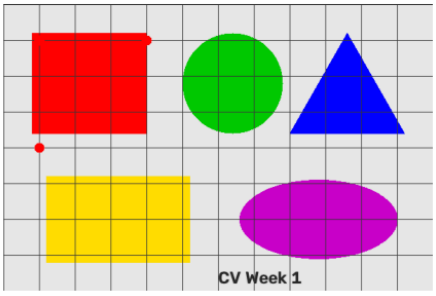
After warpPerspective = 'scanned'



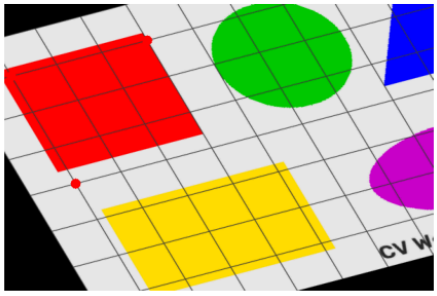
Why the lecture's perspective output is a plain grey square: its corner points (56,65)… (389,390) cover only a small patch in the top-left of a 5472×3648 photo, which is empty grey background. The code works, but the points weren't placed on an object. The 4 source points have to sit on the corners of the thing you want to flatten.

Affine vs perspective (classic eval question)

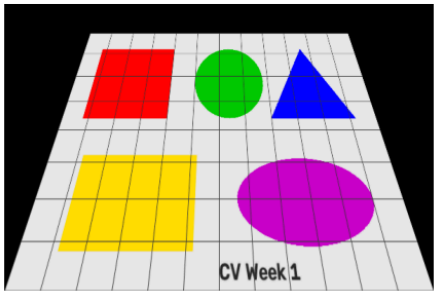
Original + grid (red = 3 source pts)



AFFINE: grid lines stay parallel



PERSPECTIVE: vertical lines now meet



	Affine	Perspective
Points needed	**3**	**4**
Matrix size	**2×3**	**3×3**
Get matrix	`cv2.getAffineTransform`	`cv2.getPerspectiveTransform`
Apply	`cv2.warpAffine`	`cv2.warpPerspective`
Parallel lines	**stay parallel**	can **meet**
Rectangle can become	parallelogram	any 4-sided shape (e.g. trapezoid)
Includes	translate, scale, rotate, shear	all affine + "tilt / viewpoint change"

▶ Run `code/day2_01_translate_scale_rotate.py` and `code/day2_02_affine_vs_perspective.py`.

---

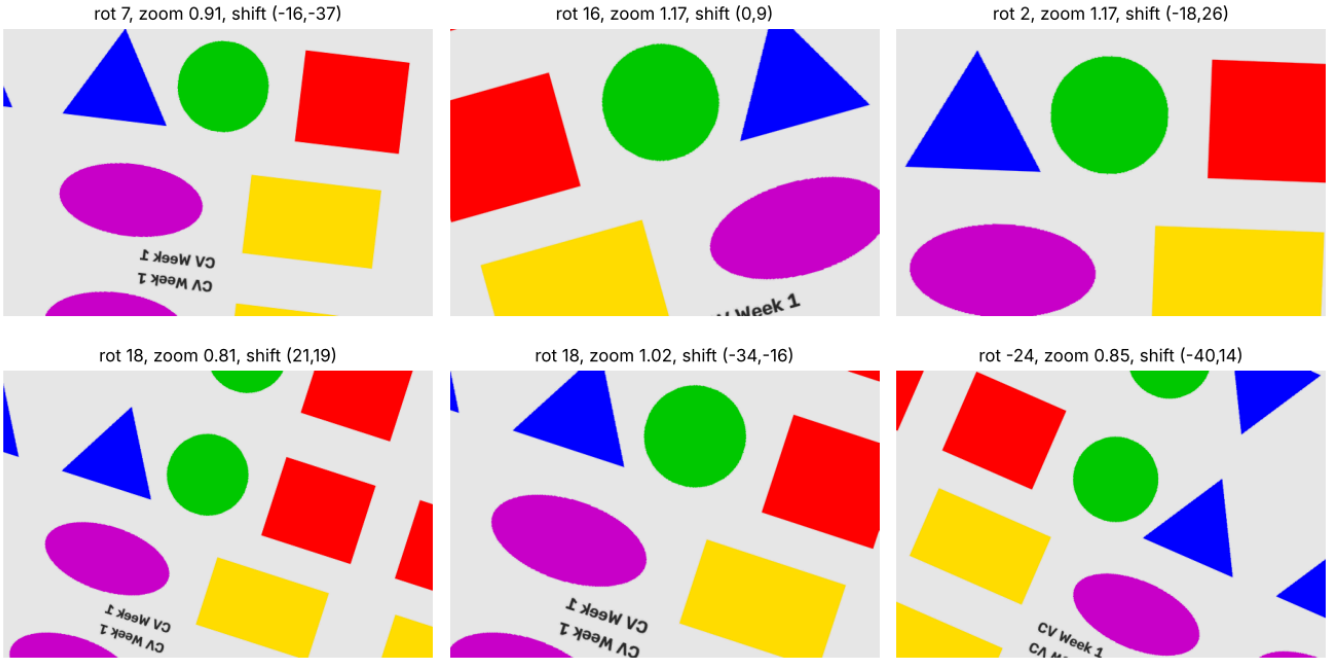
6. Mini-project: Image Augmentation Tool

What is data augmentation? AI models learn from examples, and you rarely have enough photos. So you create **extra versions** of each one: shifted, rotated, zoomed, flipped. It's still the same cat, so the model learns that a cat is a cat **whatever its position, size or angle**. More variety means the model generalises better and overfits less.

The lecture's `transform_image(img_path, mode)` uses `if / elif` to pick one transform:

mode	does
`"translate"`	shift 100 right, 50 down
`"scale"`	resize to 50%
`"rotate"`	45° around the centre
`"perspective"`	4-point warp to 300×300

`day2_03_mini_project_augmentation.py` also has a **random augmenter** (random angle, zoom, shift and flip), which is how real deep-learning pipelines do it.



✅ Day 2: check yourself

1. Write the translation matrix for "move 30 px left and 20 px down". *(`[[1,0,-30], [0,1,20]`)*
2. What does the third argument of `getRotationMatrix2D` do? *(scale; 1 = same size)*
3. Is a positive angle clockwise or anti-clockwise? *(anti-clockwise)*
4. How many points do affine and perspective need? *(3 and 4)*
5. Which interpolation is best for shrinking? *(`INTER_AREA`)*
6. Why do corners disappear after rotation? *(the output canvas keeps the original size)*

Day 3 · Filtering & Convolutions

Lecture covers: Why filtering matters · Convolution basics · Common filters (blur, sharpen, edge detection) · Canny edge detection · Applications · Lab exercise · Mini-project: Edge-based object detector

Code for today: `code/day3_01` → `day3_05`. **Start with** `day3_01_convolution_by_hand.py`: it's the key to the whole day.

---

1. Why filtering matters

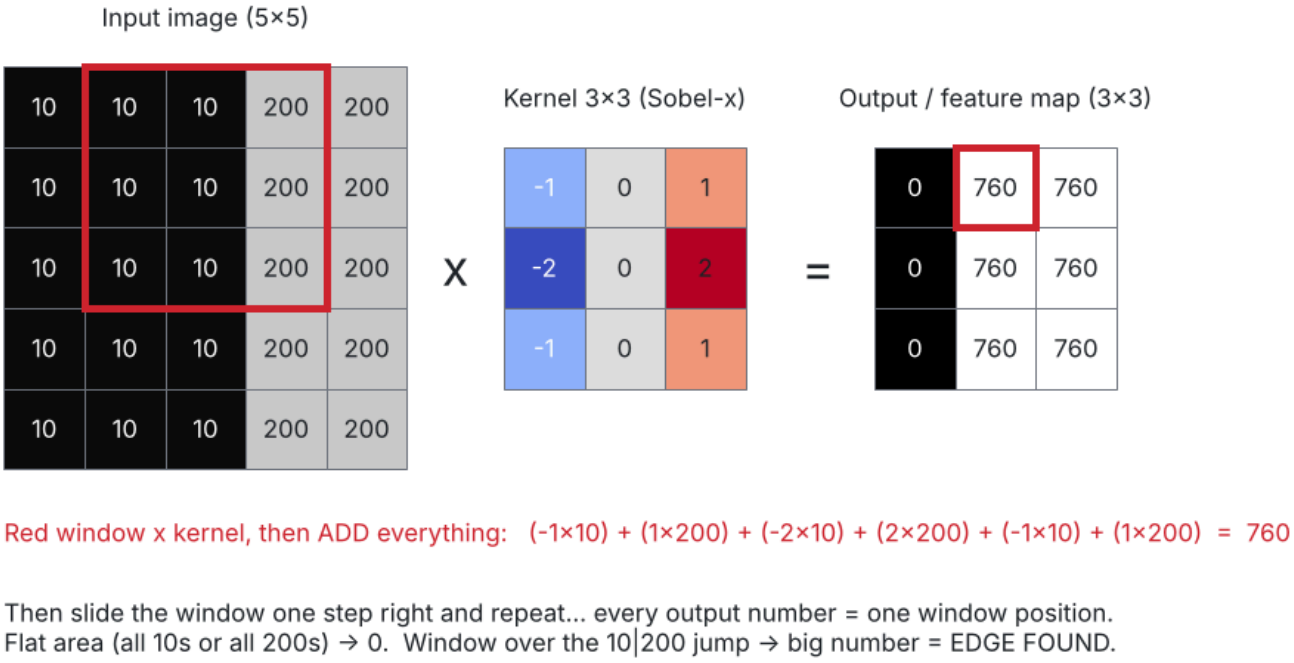
A **filter** changes each pixel based on **its neighbours**. Filters can:

- **Suppress** features: blur away noise or tiny details.
- **Enhance** features: sharpen, or highlight edges.
- They are the **foundation of CNNs**. A CNN is basically a stack of filters whose numbers the computer *learns by itself* instead of a human choosing them.

---

2. Convolution: the core idea

> **Kernel** = a small grid of numbers (usually 3×3 or 5×5). **Convolution** = slide the kernel over the image. At each spot, **multiply** each kernel number by the pixel under it, **add** them all up, and write that one number into the output. The output image is called a **feature map**: it highlights wherever the image matched the kernel's pattern.



Picture a magnifying glass with numbers written on it. You slide it across a photo one pixel at a time. At every spot you calculate one "score" for the patch underneath. Those scores form the new image.

Worked example (from the picture): window `[[10,10,200],[10,10,200],[10,10,200]]` × Sobel-x kernel `[[-1,0,1],[-2,0,2],[-1,0,1]]`:

```
(-1×10) + (0×10) + (1×200)
(-2×10) + (0×10) + (2×200)
(-1×10) + (0×10) + (1×200)   = -10+200 -20+400 -10+200 = 760   ← big number = EDGE
```

Over a flat area (all 10s): −10 + 10 − 20 + 20 − 10 + 10 = **0**, so no edge.

The kernel-sum rule (easy marks)

Kernel numbers add up to…	Effect	Examples
1	Overall brightness kept	Blur, sharpen, identity
0	Flat areas → 0 (black); only *changes* survive	Edge detectors (Sobel, Laplacian)
>1 / <1	Image gets brighter / darker	

In OpenCV

```
kernel = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])
sharp  = cv2.filter2D(img, -1, kernel)     # -1 = output has the same depth/type as
the input
```

Output size, padding, stride (CNN basics, often asked)

Without padding the kernel can't sit centred on border pixels, so the output **shrinks**: a 6×6 image with a 3×3 kernel gives 4×4. `cv2.filter2D` **pads** the border, so its output is the same size.

Formula: `output = (N − K + 2P) / S + 1` N = input size, K = kernel size, P = padding, S = stride (how many pixels the kernel jumps each step). Example: N = 6, K = 3, P = 0, S = 1 → (6 − 3 + 0)/1 + 1 = **4**.

Fine print: strictly speaking `filter2D` computes **correlation**. True convolution flips the kernel 180° first. For symmetric kernels (blur, sharpen) the results are identical. CNNs also use correlation and still call it "convolution".

▶ Run `code/day3_01_convolution_by_hand.py`. It does all of this with plain loops and proves it matches OpenCV.

---

3. Common filters

Same sliding-window maths, different numbers in the kernel → different effect

Identity (sum = 1)

0	0	0
0	1	0
0	0	0

Box blur (sum = 1)

0.11	0.11	0.11
0.11	0.11	0.11
0.11	0.11	0.11

Sharpen (sum = 1)

0	-1	0
-1	5	-1
0	-1	0

Sobel X (sum = 0)

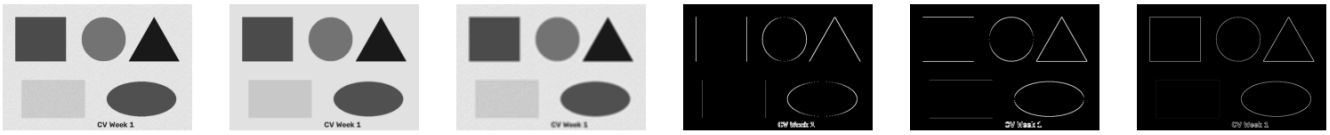
-1	0	1
-2	0	2
-1	0	1

Sobel Y (sum = 0)

-1	-2	-1
0	0	0
1	2	1

Laplacian (sum = 0)

0	1	0
1	-4	1
0	1	0



Blur (smoothing): removes noise and fine detail

Filter	Code	How it works
Average (box)	`cv2.blur(img, (5,5))`	Every pixel = plain average of its 5×5 neighbourhood. Kernel is all 1/25
Gaussian	`cv2.GaussianBlur(img, (5,5), 0)`	Weighted average: **closer neighbours count more** (bell-curve weights). Looks more natural. `0` = let OpenCV choose sigma (spread) from the kernel size
Median *(bonus)*	`cv2.medianBlur(img, 5)`	Takes the middle value. Best for "salt-and-pepper" dots

- The kernel size must be **odd** (3, 5, 7…) so there is a centre pixel.
- **Bigger kernel = more blur.**
- Blurring is often done **before** edge detection so that noise isn't mistaken for edges.

Sharpen

```
kernel = np.array([[ 0, -1,  0],
                   [-1,  5, -1],
                   [ 0, -1,  0]])
```

"5 × me minus my 4 neighbours." In a flat area 5x − 4x = x, so nothing changes. At an edge the difference between neighbours gets **exaggerated**, so it looks crisper. The sum is 1, so brightness is kept.

Edge detection: Sobel and Laplacian

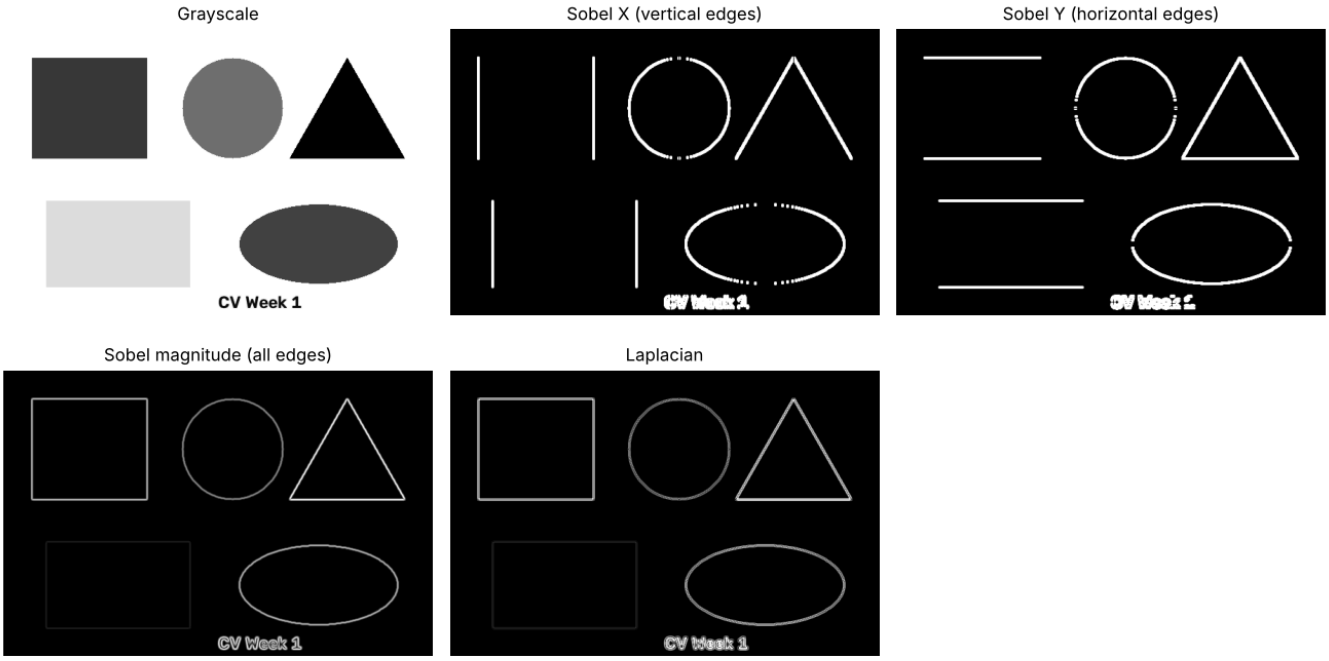
An **edge** is a place where brightness **changes suddenly**: the outline of an object.

Sobel measures how fast brightness changes in one direction (the *gradient*, i.e. the slope):

```
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=5)   # dx=1 → change left→right → VERTICAL edges
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=5)   # dy=1 → change top→bottom → HORIZONTAL edges
magnitude = np.sqrt(sobel_x**2 + sobel_y**2)           # all edges combined
```

- **Why `cv2.CV_64F`?** Dark→bright gives a positive value and bright→dark gives a **negative** one. `uint8` can't store negatives (they'd be cut to 0) and you'd lose half the edges. So compute in float, then `cv2.convertScaleAbs(...)` to take the absolute value and convert back to 0–255 for display.
- **The lecture's "Clipping input data to the valid range for imshow with RGB" warning:** it ran Sobel on the *colour* image and showed the float result (with negatives and values above 255) directly. That's why its Sobel picture looks rainbow-noisy. Fix: run Sobel on **grayscale** and use `convertScaleAbs`.

Laplacian looks at the *second* derivative (the change of the change) and finds edges in **all directions with one kernel** `[[0,1,0],[1,-4,1],[0,1,0]]`. It's very sensitive to noise, so blur first. `cv2.Laplacian(gray, cv2.CV_64F)`.



▶ Run `code/day3_02_common_filters.py`.

---

4. Canny edge detection

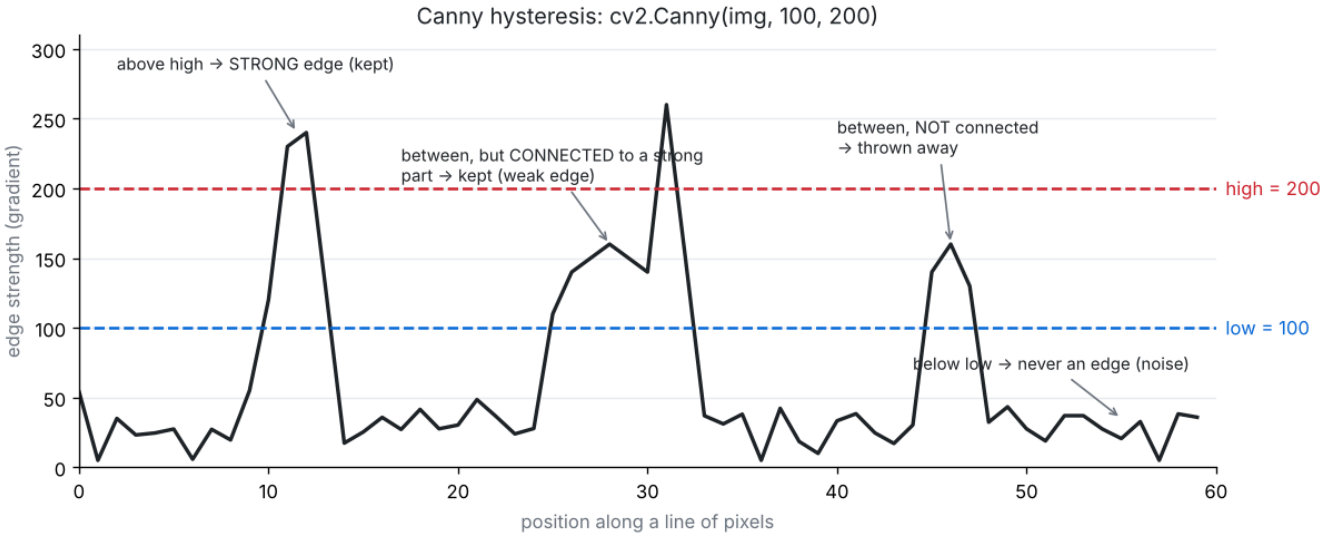
> "Combines gradient + thresholding. Produces clean edge maps."

The Canny algorithm has **4 steps**. Know them in order:

1. **Noise reduction**: Gaussian blur (5×5). *Fine print:* OpenCV's `cv2.Canny` function actually skips this step, which is why people usually call `cv2.GaussianBlur` themselves before

Canny. For the exam, list all 4 steps.

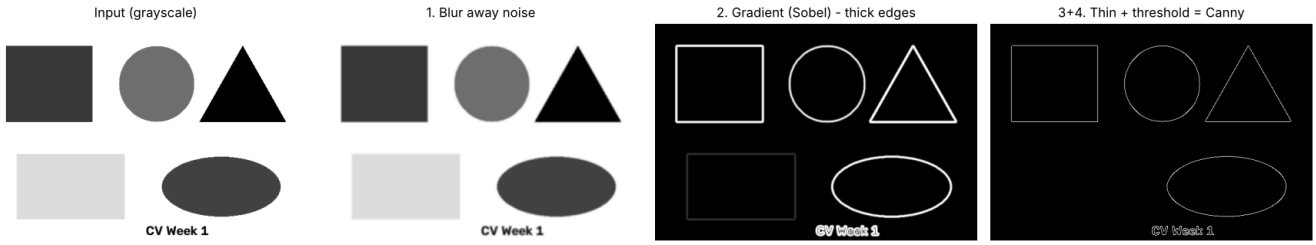
2. **Gradient**: Sobel in x and y gives the edge strength and direction at every pixel.
3. **Non-maximum suppression**: thick edges are thinned to **1 pixel wide** by keeping only the strongest pixel across each edge.
4. **Hysteresis thresholding** with the two numbers (low = 100, high = 200):



Edge strength	Decision
> high (200)	**Strong edge**: keep
< low (100)	Not an edge: discard
between	Keep **only if connected to a strong edge**, otherwise discard

This "connected" rule is what makes Canny's lines clean and continuous instead of broken and noisy.

- Output: a **single-channel, black image with white 1-pixel lines** (values only 0 or 255).
- **Lower thresholds** give more edges (including noise and texture). **Higher thresholds** keep only strong outlines. Rule of thumb: high ≈ 2–3 × low.



Why the lecture's Canny images look almost black: the photo is huge (5472×3648) and soft-focused. The edges are only 1 pixel thin, so when such a big image is shrunk to fit on the page most of them disappear. Resize the image first or lower the thresholds.

▶ Run `code/day3_03_canny.py`.

---

5. Applications of filtering

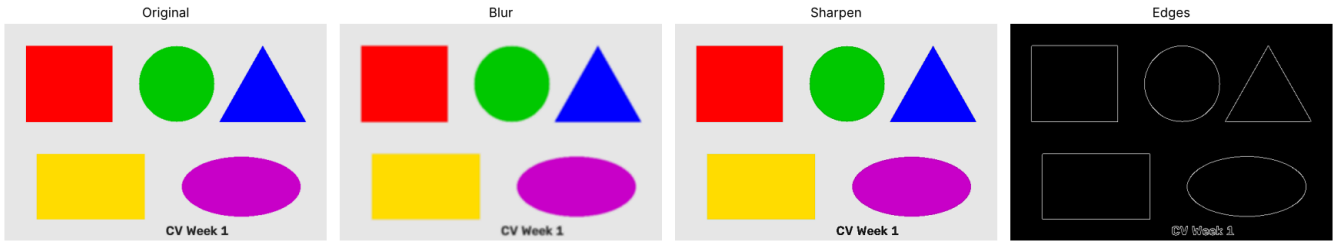
Area	Use
Medical imaging	Find **tumour boundaries** (edges)
Document scanning	Find the page's edges and clean up/sharpen the text
Autonomous driving	**Lane detection** (edges of the lane markings)

---

6. Lab exercise: compare filters side by side

```
filters = {
    "blur":    cv2.blur(img, (5,5)),
    "sharpen": cv2.filter2D(img, -1, kernel),
    "edges":   cv2.Canny(img, 50, 150),
}
for name, output in filters.items():
    plt.figure()
    plt.imshow(output if name=="edges" else cv2.cvtColor(output, cv2.COLOR_BGR2RGB),
               cmap='gray' if name=="edges" else None)
    plt.title(name.capitalize())
```

Why the `if name == "edges"`? Canny returns a **1-channel** image, which can't be converted BGR→RGB and needs `cmap='gray'`. Blur and sharpen return 3-channel BGR images, which need converting.



---

7. Mini-project: Edge-based object detector

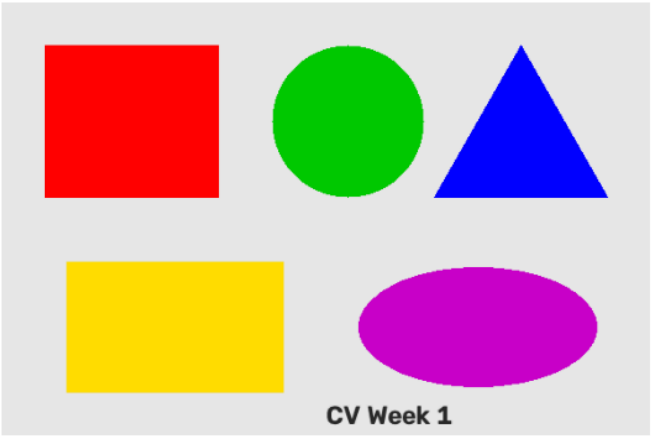
Pipeline: colour → grayscale → Canny edges → **find contours** → draw boxes and labels.

```
gray  = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
for i, cnt in enumerate(contours):
    x, y, w, h = cv2.boundingRect(cnt)
    cv2.rectangle(img_contours, (x,y), (x+w,y+h), (0,255,0), 2)
    cv2.putText(img_contours, f"Obj{i+1}", (x,y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
(0,255,0), 1)
```

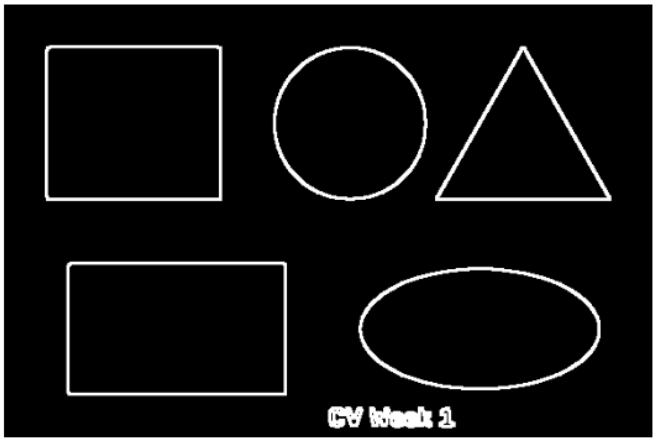
Piece	Plain meaning
Contour	A list of points tracing the outline of one shape (like drawing around a coin with a pencil)
`findContours` returns	`(contours, hierarchy)`. Hierarchy says which outline is inside which. The lecture ignores it with `_`
`RETR_EXTERNAL`	Only the **outermost** outlines (ignore holes and inner shapes). `RETR_TREE` = all of them, with nesting
`CHAIN_APPROX_SIMPLE`	Store only the **corner points** of straight runs. A rectangle becomes 4 points instead of hundreds
`boundingRect(cnt)`	The smallest **upright box** around the contour: `x, y` = top-left corner, `w, h` = size
`rectangle(img, pt1, pt2, colour, thickness)`	Draws the box. `(0,255,0)` in BGR = **green**
`putText(img, text, position, font, scale, colour, thickness)`	Writes the label just above the box (`y-10`)

Why the lecture's result has green boxes all over the text and noise: every tiny squiggle Canny finds becomes a "contour". Standard fixes (all in `day3_05_...py`): 1. **Blur first** (`GaussianBlur`) so there's less noise. 2. **Ignore small contours**: `if cv2.contourArea(cnt) < 500: continue`. 3. **Dilate** the edges to close tiny gaps so each shape is one closed outline. 4. *Bonus:* name each shape by counting corners with `cv2.approxPolyDP`: 3 = triangle, 4 = rectangle/square, many = circle.

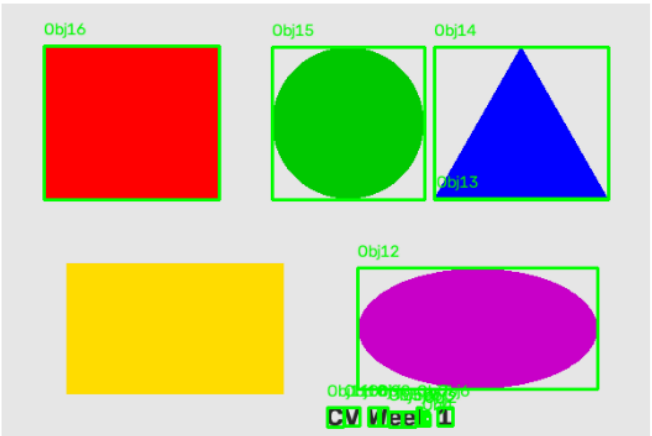
Input: simple shapes



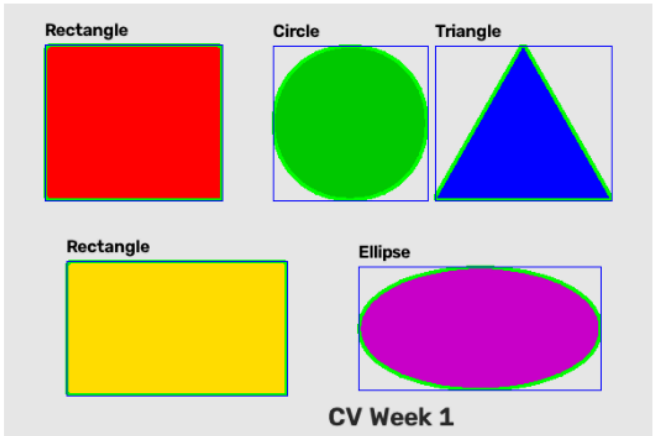
Edges (blur + Canny + dilate)



Lecture version: 16 'objects'



Improved: 5 labelled shapes



Notice the yellow rectangle: with the lecture's thresholds (50, 150) Canny **misses it**. Yellow (grey value 205) on light grey (230) is only 25 brightness levels apart. **Edges are found on brightness, not colour**, so two different colours with similar brightness look the same to an edge detector.

▶ Run `code/day3_05_mini_project_object_detector.py`.

✅ Day 3: check yourself

1. In one sentence, what is convolution? *(Slide a small kernel over the image and, at each spot, multiply and add to get one output pixel.)*
2. Sharpen kernel sum? Sobel kernel sum? What does each mean? *(1 keeps brightness; 0 makes flat areas black)*
3. Why `cv2.CV_64F` in Sobel? *(edge values can be negative)*
4. The 4 steps of Canny? *(blur → gradient → non-max suppression → hysteresis)*
5. In `Canny(img, 100, 200)`, what happens to a pixel with strength 150? *(kept only if connected to a strong edge, i.e. one above 200)*
6. Output size for a 32×32 input, 5×5 kernel, no padding, stride 1? *((32−5)/1+1 = 28 → 28×28)*
7. What does `RETR_EXTERNAL` do? *(returns only the outermost contours)*

Week 1 Practice Quiz

Answer each question **out loud or on paper first**, then click ▸ to check. Questions are in the styles evals usually use: concepts, "what does this code do", "predict the output", and "find the bug".

Part A · Concepts (Day 1)

A1. What is Computer Vision? Give 3 applications.

> **Answer:** The field of AI that lets machines interpret and understand visual information (images and video). Applications: self-driving cars, medical imaging (tumour detection), surveillance, augmented reality, traffic monitoring, retail (face recognition, product tracking), creative AI (art generation).

A2. What is a pixel? What values can a pixel in an 8-bit grayscale image have?

> **Answer:** The smallest square ("picture element") of a digital image. In an 8-bit grayscale image each pixel is one whole number from 0 (black) to 255 (white), because 8 bits give 2^8 = 256 values.

A3. An image has `img.shape == (720, 1280, 3)`. What are its width, height and number of channels? How many numbers does it store in total?

> **Answer:** Height 720, width 1280, 3 channels. 720 × 1280 × 3 = 2,764,800 numbers.

A4. What does "resolution" mean and why does it matter?

> **Answer:** The number of pixels (width × height). More pixels = more detail, but also more memory and slower processing.

A5. In which order does OpenCV store colour channels? What goes wrong if you ignore this?

> **Answer:** BGR (Blue, Green, Red). Matplotlib expects RGB, so if you show a cv2 image with plt.imshow without converting, red and blue swap. Fix: `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.

A6. What do H, S and V stand for? What is the range of H in OpenCV and why?

> **Answer:** Hue (which colour), Saturation (how vivid/pure), Value (how bright). OpenCV's H is 0–179: the real range is 0–360°, and halving it lets it fit in a uint8 (max 255).

A7. Why is HSV preferred over RGB for colour detection?

> **Answer:** HSV keeps the colour (Hue) separate from the brightness (Value). Lighting changes like shadows mostly change V, so you can detect a colour with a Hue range that works under different lighting. In RGB, lighting changes all three numbers.

A8. Grayscale formula used by OpenCV? Why does green get the biggest weight?

> **Answer:** Y = 0.299R + 0.587G + 0.114B. Human eyes are most sensitive to green light.

A9. What is a mask?

> **Answer:** A single-channel black/white image of the same size as the original: 255 (white) = keep/selected pixel, 0 (black) = ignore. Made with e.g. `cv2.inRange` and used with `cv2.bitwise_and`.

Part B · Concepts (Day 2)

B1. Write the translation matrix that moves an image 40 px right and 25 px up.

> **Answer:** `np.float32([[1, 0, 40], [0, 1, -25]])`. Up is a negative y because y grows downwards.

B2. In `cv2.getRotationMatrix2D(center, angle, scale)`, what does each argument do? Which direction is a positive angle?

> **Answer:** center = pivot point (x, y); angle in degrees; scale = resize factor (1 = same size). Positive angle = anti-clockwise.

B3. Why do the corners of an image get cut off after rotating by 45°?

> **Answer:** The output size given to warpAffine is the same as the original (cols, rows), but a rotated rectangle needs a bigger box, so the corners fall outside the canvas. Empty areas become black.

B4. What is the defining property of an affine transformation? How many points define it?

> **Answer:** Parallel lines stay parallel (and straight lines stay straight). It's defined by 3 point pairs, giving a 2×3 matrix.

B5. How is a perspective transformation different from an affine one? Give one real-world use.

> **Answer:** Perspective maps any quadrilateral to any quadrilateral using 4 point pairs and a 3×3 matrix; parallel lines can converge. Used for document scanning (flattening a page photographed at an angle), bird's-eye view of a road, and so on.

B6. Which interpolation would you choose to shrink an image? To enlarge it smoothly?

> **Answer:** Shrink: `cv2.INTER_AREA`. Enlarge smoothly: `cv2.INTER_CUBIC` (or INTER_LINEAR, the default, which is faster).

B7. What is data augmentation and why is it used?

> **Answer:** Creating modified copies of training images (shifted, rotated, scaled, flipped, warped…) so a model sees more variety. This reduces overfitting and makes the model robust to position, size and angle changes.

B8. Translation, scaling and rotation: are they affine?

> **Answer:** Yes. All three (and shear) are special cases of affine transformations.

Part C · Concepts (Day 3)

C1. Define kernel, convolution and feature map.

> **Answer:** Kernel: a small matrix of weights (e.g. 3×3). Convolution: sliding the kernel over the image and at each position computing the weighted sum (multiply element-wise, then add). Feature map: the output image of those sums, which highlights where the pattern occurs.

C2. Compute the output of this 3×3 patch with the sharpen kernel (centre value only):

```
patch = [[10, 10, 10],        kernel = [[ 0,-1, 0],
         [10, 50, 10],                  [-1, 5,-1],
         [10, 10, 10]]                  [ 0,-1, 0]]
```

> **Answer:** 5×50 − 10 − 10 − 10 − 10 = 250 − 40 = **210**. The bright centre gets even brighter compared with its neighbours: sharpening.

C3. Same patch, with the 3×3 box (average) blur kernel?

> **Answer:** (8×10 + 50)/9 = 130/9 ≈ **14.4**. The bright spot gets smoothed away.

C4. Kernel sums: what do they tell you? Give the sum for blur, sharpen and Sobel.

> **Answer:** Sum 1 keeps overall brightness (blur = 1, sharpen = 1). Sum 0 sends flat regions to 0, keeping only changes, i.e. edges (Sobel = 0, Laplacian = 0).

C5. Difference between average blur and Gaussian blur?

> **Answer:** Average blur weights all neighbours equally. Gaussian blur weights them with a bell curve, so closer pixels count more. Gaussian looks more natural and preserves edges a bit better.

C6. Why must kernel sizes like (5, 5) be odd?

> **Answer:** So the kernel has a single centre pixel to place on the pixel being computed.

C7. What do `dx=1, dy=0` in `cv2.Sobel` detect? Why is `cv2.CV_64F` used?

> **Answer:** Changes in the x direction, which means **vertical** edges. CV_64F (float) is used because gradients can be negative (bright→dark). uint8 would clip negatives to 0 and lose those edges.

C8. List the 4 stages of the Canny edge detector.

> **Answer:** 1) Gaussian noise reduction, 2) gradient computation (Sobel: magnitude + direction), 3) non-maximum suppression (thin edges to 1 px), 4) hysteresis thresholding with low/high thresholds.

C9. In `cv2.Canny(img, 100, 200)`, classify pixels with gradient strength 250, 150 and 50.

> **Answer:** 250 → strong edge (kept). 150 → weak, kept only if connected to a strong edge. 50 → discarded.

C10. What happens to the Canny result if you lower both thresholds?

> **Answer:** More edges are detected, including noise and texture. Raising them keeps only the strongest outlines.

C11. Sobel vs Laplacian?

> **Answer:** Sobel is a first derivative, computed separately in x or y (it has a direction). Laplacian is a second derivative that detects edges in all directions with one kernel, but it is more sensitive to noise.

C12. Why is convolution called "the foundation of CNNs"?

> **Answer:** CNN layers are made of convolution kernels. Instead of hand-designing kernels (like Sobel), the network learns the kernel values from data. Early layers end up learning edge-like filters.

C13. Output size of a 28×28 image convolved with a 3×3 kernel, stride 1, no padding? With padding 1?

> **Answer:** $(28 - 3 + 0)/1 + 1 = 26 \rightarrow 26\times26$. With $P = 1$: $(28 - 3 + 2)/1 + 1 = 28 \rightarrow 28\times28$ (same size).

C14. What is a contour? What do `RETR_EXTERNAL` and `CHAIN_APPROX_SIMPLE` mean?

> **Answer:** A contour is a curve of points along the boundary of a shape. RETR_EXTERNAL returns only the outermost contours. CHAIN_APPROX_SIMPLE compresses straight segments down to their end points to save memory.

C15. The edge-based object detector draws boxes around lots of tiny noise. Give 2 fixes.

> **Answer:** Blur before Canny; skip contours with `cv2.contourArea(cnt) < threshold`; dilate the edges to close gaps; tune the Canny thresholds.

C16. Name one application of filtering in each of medical imaging, document scanning and autonomous driving.

> **Answer:** Tumour boundary detection; detecting page edges / sharpening text; lane detection.

Part D · Predict the output

D1.

```
img = cv2.imread('photo.jpg')     # a 400 × 600 colour photo (height 400)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
print(gray.shape)
```

> **Answer:** `(400, 600)`. Grayscale has no channel dimension.

D2.

```
small = cv2.resize(img, None, fx=0.5, fy=0.25)
print(small.shape)               # img is (400, 600, 3)
```

> **Answer:** `(100, 300, 3)`. Width 600×0.5 = 300, height 400×0.25 = 100.

D3.

```
a = np.array([200], dtype=np.uint8)
print(a + a)
```

> **Answer:** `[144]`. 400 wraps around: 400 − 256 = 144. (`cv2.add` would give 255.)

D4.

```
edges = cv2.Canny(img, 100, 200)
print(edges.shape, np.unique(edges))      # img is (400, 600, 3)
```

> **Answer:** `(400, 600) [ 0 255]`, a single-channel image containing only 0 and 255 (assuming some edges were found).

D5.

```
img = np.zeros((100, 200, 3), np.uint8)
img[:, :100] = (0, 0, 255)
print(img[50, 150], img[50, 50])
```

> **Answer:** `[0 0 0] [ 0 0 255]`. The left half (columns 0–99) is red in BGR. Pixel (row 50, col 150) is on the right, so it's black.

D6.

```
hsv = cv2.cvtColor(np.uint8([[[0, 255, 0]]]), cv2.COLOR_BGR2HSV)
print(hsv)
```

> **Answer:** `[[[ 60 255 255]]]`. Pure green has hue 120°, which OpenCV halves to 60.

Part E · Find the bug

E1.

```
img = cv2.imread('sample.jpg')
plt.imshow(img)          # colours look wrong
```

> **Answer:** BGR shown as RGB. Use `plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))`.

E2.

```
M = np.float32([[1, 0, 100], [0, 1, 50]])
translated = cv2.warpAffine(img, M, (rows, cols))
```

> **Answer:** The size must be (width, height) = `(cols, rows)`. Also `rows, cols` must be defined first: `rows, cols = img.shape[:2]`.

E3.

```
blur = cv2.GaussianBlur(img, (4, 4), 0)
```

> **Answer:** The kernel size must be odd, e.g. (3, 3) or (5, 5). An even size raises an error.

E4.

```
sobel = cv2.Sobel(gray, cv2.CV_8U, 1, 0)
```

> **Answer:** CV_8U can't hold negative gradients, so bright→dark edges are lost. Use `cv2.CV_64F` and then `cv2.convertScaleAbs`.

E5.

```
img = cv2.imread('sampel.jpg')
print(img.shape)          # AttributeError: 'NoneType' object has no attribute
'shape'
```

> **Answer:** The file name is misspelled or the path is wrong. `cv2.imread` doesn't raise an error; it returns `None`. Always check `if img is None`.

E6.

```
M = cv2.getPerspectiveTransform(pts1, pts2)
out = cv2.warpAffine(img, M, (300, 300))
```

> **Answer:** A perspective matrix is 3×3 and must be applied with `cv2.warpPerspective`.

E7.

```
edges = cv2.Canny(img, 100, 200)
plt.imshow(cv2.cvtColor(edges, cv2.COLOR_BGR2RGB))
```

Answer: Canny returns a single-channel image, so a BGR→RGB conversion makes no sense: there are no colour channels to swap. In OpenCV 4.x it raises an "Invalid number of channels" error; newer versions quietly copy the grey into 3 channels, which is pointless. Use `plt.imshow(edges, cmap='gray')`.

CV Week 1 · Cheat Sheet

Images = numbers

- Image = grid of **pixels**. `uint8` → values **0 (black) – 255 (white)**.
- `img.shape = (height, width, channels)` = `(rows, cols, 3)`. Grayscale: `(rows, cols)`.
- Pixel access: **`img[y, x]`** (row first). Origin (0,0) = **top-left**, y grows **down**.
- OpenCV size arguments are **(width, height)** = `(cols, rows)`.
- `cv2.imread` returns **`None`** if the path is wrong (no error!).
- uint8 overflow: NumPy wraps (250+10 = 4); `cv2.add` saturates (= 255).

Colour

OpenCV order	**BGR**. Matplotlib wants RGB → `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`
Gray	`cv2.COLOR_BGR2GRAY`; Y = 0.299R + 0.587G + 0.114B
HSV	`cv2.COLOR_BGR2HSV`. **H** = which colour (**0–179**), **S** = vividness, **V** = brightness (0–255)
Why HSV	colour is separate from brightness → robust colour detection
Mask	`cv2.inRange(hsv, low, high)` → 255 inside the range; `cv2.bitwise_and(img, img, mask=m)`
Hues	red 0/179 · orange ~15 · yellow ~30 · green ~60 · blue ~120

Transformations (Day 2) — `rows, cols = img.shape[:2]`

Task	Code	Notes
Translate	`M = np.float32([[1,0,tx],[0,1,ty]])`; `cv2.warpAffine(img, M, (cols, rows))`	+tx right, +ty **down**
Scale	`cv2.resize(img, None, fx=.5, fy=.5, interpolation=cv2.INTER_LINEAR)`	shrink: `INTER_AREA`; enlarge: `INTER_CUBIC`
Rotate	`M = cv2.getRotationMatrix2D((cols/2, rows/2), angle, scale)` + `warpAffine`	**+angle = anti-clockwise**; corners get cut
Affine	`cv2.getAffineTransform(pts1, pts2)` + `warpAffine`	**3 points**, 2×3 matrix, **parallel lines stay parallel**
Perspective	`cv2.getPerspectiveTransform(pts1, pts2)` + **`warpPerspective`**	**4 points**, 3×3 matrix, parallel lines can meet → **document scanning**
Flip / 90°	`cv2.flip(img, 1)` · `cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)`	

Augmentation = making extra training images by transforming them → more variety, less overfitting.

Convolution (Day 3)

- **Kernel** = small matrix. **Convolution** = slide it over the image, **multiply element-wise and sum** at each spot. Output = **feature map**. It's the **foundation of CNNs** (which learn their own kernels).
- `cv2.filter2D(img, -1, kernel)` (-1 = same depth as the input). Technically correlation (no kernel flip).
- **Kernel sum = 1** → brightness kept (blur, sharpen). **Sum = 0** → edge detector (flat → 0).
- Output size: **(N − K + 2P)/S + 1**.
- Kernel sizes must be **odd**. Bigger kernel = stronger effect.

Filter	Code / kernel
Average blur	`cv2.blur(img, (5,5))`, all weights equal
Gaussian blur	`cv2.GaussianBlur(img, (5,5), 0)`, closer = heavier weight
Median	`cv2.medianBlur(img, 5)`, salt & pepper noise
Sharpen	`[[0,-1,0],[-1,5,-1],[0,-1,0]]`
Sobel	`cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=5)` → dx=1: **vertical** edges; dy=1: horizontal. `[[-1,0,1],[-2,0,2],[-1,0,1]]`
Laplacian	`cv2.Laplacian(gray, cv2.CV_64F)`, 2nd derivative, all directions, noise-sensitive. `[[0,1,0],[1,-4,1],[0,1,0]]`
Display float	`cv2.convertScaleAbs(result)` → abs + uint8

`CV_64F` because gradients can be **negative**; uint8 would lose them.

Canny — `cv2.Canny(img, low, high)`

1. Gaussian blur (noise) → 2. Sobel gradient (strength + direction) → 3. **Non-max suppression** (1-px thin) → 4. **Hysteresis**: **> high** = strong edge · **< low** = discard · **between** = keep **only if connected to a strong edge**.
2. Output: 1-channel, values **0/255** only. Lower thresholds → more edges/noise. high ≈ 2–3 × low.

Contours (object detector)

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(cv2.GaussianBlur(gray, (5,5), 0), 50, 150)
contours, hierarchy = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
for cnt in contours:
    if cv2.contourArea(cnt) < 500: continue          # skip noise
    x, y, w, h = cv2.boundingRect(cnt)
    cv2.rectangle(img, (x, y), (x+w, y+h), (0,255,0), 2)
    cv2.putText(img, "Obj", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,255,0), 1)
```

`RETR_EXTERNAL` = outer outlines only · `CHAIN_APPROX_SIMPLE` = keep corner points only · `approxPolyDP` corner count → shape name.

Applications

CV: self-driving cars, medical imaging, surveillance, AR, traffic monitoring, retail (faces, products), creative AI. Filtering: tumour boundaries, document scanning, lane detection.

Display gotchas

- `plt.imshow(gray, cmap='gray')` for 1-channel images (otherwise you get fake colours).
- Colour from cv2 → convert BGR→RGB first. `cv2.imshow` needs no conversion.